Protected Routes & Authentication Flow

3.2 – Protected Routes & Authentication Flow

🎯 What Interviewers Actually Ask

Question	Level	Frequency
"What are protected/private routes?"	🟡	★★★★☆
"How do you implement route guards in React?"	🟡🔴	★★★★☆
"How do you handle authentication in a React app?"	🟡🔴	★★★★★
"What is the difference between authentication and authorization?"	🟡	★★★★☆
"How do you implement role-based access control?"	🔴	★★★☆☆

Authentication answers: "Who are you?" (verifying identity through login).

Authorization answers: "What are you allowed to do?" (checking permissions and roles).

A protected route deals with both. It first checks if the user is authenticated (logged in), then optionally checks if they're authorized (have the right role/permissions).

Building a Protected Route Component

The pattern is straightforward: wrap your route in a component that checks authentication status. If the user is logged in, render the child. If not, redirect to login.

JSX Copy

```jsx
import { Navigate, useLocation } from 'react-router-dom';

function ProtectedRoute({ children }) {
  const { user } = useAuth();  // Your auth hook/context
  const location = useLocation();

  if (!user) {
    // Redirect to login, but save where the user was trying to go
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  return children;
}
```

Using it in your routes

JSX Copy

```jsx
function App() {
  return (
    <BrowserRouter>
      <Routes>
        {/* Public routes */}
```

```
        {/* Protected routes */}
        <Route
          path="/dashboard"
          element={
            <ProtectedRoute>
              <Dashboard />
            </ProtectedRoute>
          }
        />
        <Route
          path="/profile"
          element={
            <ProtectedRoute>
              <Profile />
            </ProtectedRoute>
          }
        />
      </Routes>
    </BrowserRouter>
  );
}
```

Redirecting Back After Login

The `state={{ from: location }}` in the Navigate component saves where the user was trying to go. After login, you can redirect them back:

JSX Copy

```
function LoginPage() {
  const navigate = useNavigate();
  const location = useLocation();
  const { login } = useAuth();

  const from = location.state?.from?.pathname || '/dashboard';
```

```
    if (success) {
      navigate(from, { replace: true });  // Go back to where they came fr
    }
  };

  return <form onSubmit={handleSubmit}> ... </form>;
}
```

Role-Based Access Control

For apps with different user roles (admin, editor, viewer), extend the protected route to check roles.

JSX Copy

```jsx
function RoleRoute({ children, allowedRoles }) {
  const { user } = useAuth();
  const location = useLocation();

  if (!user) {
    return <Navigate to="/login" state={{ from: location }} replace />;
  }

  if (!allowedRoles.includes(user.role)) {
    return <Navigate to="/unauthorized" replace />;
  }

  return children;
}
```

JSX Copy

```jsx
// Usage
<Route
  path="/admin"
```

```
    </RoleRoute>
  }
/>
<Route
  path="/editor"
  element={
    <RoleRoute allowedRoles={['admin', 'editor']}>
      <ContentEditor />
    </RoleRoute>
  }
/>
```

The Auth Context Pattern

Most apps use a Context to provide authentication state throughout the app.

JSX Copy

```jsx
const AuthContext = createContext(null);

function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Check if user is already logged in (e.g., from a stored token)
    const token = localStorage.getItem('token');
    if (token) {
      validateToken(token)
        .then(userData => setUser(userData))
        .catch(() => localStorage.removeItem('token'))
        .finally(() => setLoading(false));
    } else {
      setLoading(false);
    }
  }, []);
```

```
    setUser(response.user);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

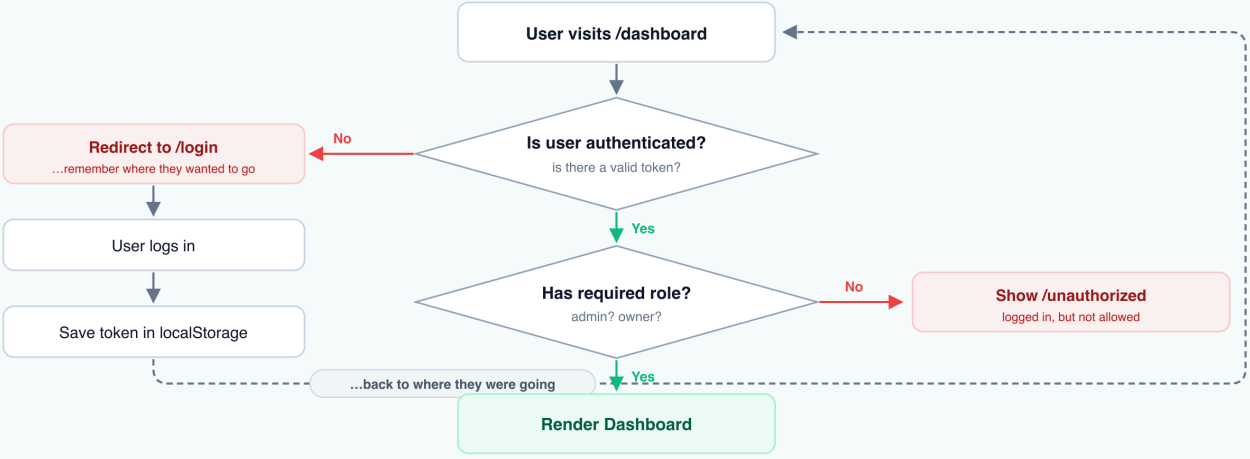
  if (loading) return <FullPageSpinner />;

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}

function useAuth() {
  const context = useContext(AuthContext);
  if (!context) throw new Error('useAuth must be used inside AuthProvider'
  return context;
}
```

The protected route flow

Authentication first (who are you?), then authorization (are you allowed?). They are two different questions.



Check authentication first, then authorization. Save the intended destination so the user lands where they meant to go, not on a generic home page.

This diagram is UI only. It stops people seeing a page; it does not protect data. Anyone can skip React and call your API directly — so the server must check the token too, every time.

Common Mistakes

Mistake 1: Storing tokens in state only. If the user refreshes the page, they get logged out. Persist tokens in localStorage or httpOnly cookies.

Mistake 2: Not showing a loading state. While checking if the user is authenticated (e.g., validating a token on mount), show a spinner. Otherwise the user briefly sees the login page before being redirected to the dashboard.

Mistake 3: Client-side auth as the only security layer. Protected routes prevent UI access, but they don't protect your data. Your API must always validate tokens independently. Someone can bypass React and hit your API directly.

Interview Answer

> "Protected routes are components that check if a user is authenticated before rendering their children. If the user isn't logged in, they redirect to the login page. I implement this with a ProtectedRoute wrapper that uses an auth context to check the user's status. For role-based access, I extend it to check the user's role against allowed roles. The key is saving the intended destination so the user gets redirected back after login. And I always emphasize that client-side route protection is for UX only. The real security must be on the API side."

Key Takeaways

- **Protected routes** check auth status before rendering. Redirect to login if unauthenticated.
- Save the **intended destination** with `location.state` so users return after login.
- **Role-based routes** extend the pattern by checking `user.role` against allowed roles.

- Use **AuthContext** to provide auth state app-wide.
- Show a **loading spinner** while validating the token on initial load.
- Client-side protection is **UX only**. API must validate tokens independently.

QUICK CHECK — no marks, just for you.

1. A protected route checks a user's role against a list of allowed roles before rendering an admin panel. Which concept is that role check an example of?

- ◯ Authorization, because it decides what the user is allowed to do
- ◯ Neither; role checks are a form of session management, not auth
- ◯ Authentication, because the role is read from the logged-in user object
- ◯ Authentication, because it verifies the user's identity

Check answer

2. In the ProtectedRoute component, the redirect is written as <Navigate to="/login" state={{ from: location }} replace />. What does the state prop accomplish here?

- ◯ It passes the auth context down to the login page so it can call login() without useAuth
- ◯ It replaces the current history entry so the back button cannot return to the protected page
- ◯ It records the page the user was trying to reach so the login page can send them back there after a successful login
- ◯ It persists the user's session across a page refresh so they are not logged out

Check answer

3. [illegible] itive page in ProtectedRoute and RoleRoute, and the routing logic is thoroughly tested. According to the lesson, why is your data still not secure?

- ○ React Router evaluates guards after the child component has already mounted and fetched data
- ○ Role checks run before authentication checks, so an unauthenticated user can slip past RoleRoute
- ○ Storing the token in localStorage means the guard cannot be trusted until the token is validated server-side on every render
- ○ Route guards only gate the UI; a client can call the API directly, so the API must validate tokens on its own

Check answer

‹ Previous
React Router: Complete Guide

Next ›
Code Splitting & Lazy Loading Routes

LEARN MORE

ARTICLES & DOCS

- React Router — Routing basics
  reactrouter.com
- React Router — home & docs
  reactrouter.com

VIDEOS



Protected routes & auth with React Router v6
YouTube

React.js Interview Kit



Private & protected routes (v6)

YouTube

Hand-picked references for this lesson.